DATA STRUCTURES SECTION D TWENTY SPRING 2026

ASSIGNMENT # 3 PART THREE

EXPLORE YOU LEARNING

Topic: The Learning Check Point

ADTs: Array, Stack, Queue and LinkedList

Total Marks = Decided by Student 😊

Assign Date: Thursday April 23, 2026

Handwritten Part per Student due in Class on Thursday April 30, 2026
Code Part per Student due on Portal 11:59 PM Wednesday April 29, 2026

😊 BEE HAPPY 😊

Finding Errors in the Assignment and Correcting these IS Part of the Assignment

CLO	Description	Taxonomy Level	PLO
2	Solve basic problems using fundamental concepts of programming language (C++).	C3 (Apply)	2

DON'T ATTEMP THE SOLUTION BEFORE CAREFUL READING OF THE ASSIGNMENT DESCRIPTION (and taking notes in the DS notebook)

MARKS ARE BASED ON EFFORT AND NOT CORRECTION

- **Cheating or Copy is Cheating Yourself out of Learning**
- **Cheating or Copy may give you undeserved marks in the assignment but you lose in quizzes, exams and life**
- Use of ChatGPT is Encouraged for help and learning but **PLEASE DON'T COPY**
- Discussion with fellow Students is Encouraged for help and learning but **PLEASE DON'T COPY**
- **So please don't copy from anyone or anywhere**

SUBMISSION DEADLINE EXTENSION POLICY

- **Extension** in the **deadline** is **possible** with some **penalty (10 – 25 marks deduction) but must** submit at least **5 Problems** by **the deadline on the portal or NO** extension and no marks

Submission of Complete Assignment by the deadline on the portal will earn 25 % bonus Marks (Bonus Marks can be redeemed for future Assignment Marks Deficiencies)

8 WEEK REFRESHER ASSIGNMENT

Purpose

The purpose of this assignment is to give you a through practice and self-assessment of your learning and coding skills of the past 8 weeks.

Style and Format

This document has problems, categorized by topic, with increasingly from first concepts (Abstract ARRAY) to (LinkedList) and tasks.

How to Do: Use the following steps to First Explore and then Improve your DS Coding:

1. **Please code these problems in SEQUENCE and IDENTIFY and NOTE DOWN the problems or portions of problems that you are stuck and can't do without help.**
2. **After completing task, please fill out the self-assessment table below (Use this table for assigning yourself marks). [Your self-assessed marks will not be accepted without this table]**
3. **Exploratory Problems: (Write a single program or multiple, but code that compiles and runs)**

Marks

1. **REVIEW and ASSESS your learning and assign yourself marks as per your learning YOURSELF.**
2. **After completing task, please fill out the self-assessment table below (Use this table for assigning yourself marks). [Your self-assessed marks will not be accepted without this table].**

Due Date(s)

1. **Please note that in this assignment You don't have to do all the problems and all the tasks**
2. **Submit as many problems (you may skip parts that you are confident that you know, but must provide a proof of the confidence in the self-assessment table below) (each problem is 05 - 10 marks) as you can do by Thursday April 30, 2026. (don't forget to assess your leaning and assign marks as per your learning and coding skills rather than what you are submitting)**

HANDWRITTEN PART: Please add a TITLE PAGE having the following information:

1. **Title: "Spring 2026 DS Section D11 Assignment THREE"**
2. **Date and Day OF Submission e.g. "Thursday April 30, 2026"**
3. **Full Registration Number e.g. "L1F23BSCS0001"**
4. **Full Name and Signature e.g. "Usman Ahmad"**
5. **Number of tasks Completed: decided BY THE STUDENT**
6. **Time taken for each Task: T1: 30 minutes T2: 60 minutes etc.**
7. **Filled the Self-Assessment Table: answer YES or NO**
8. **Self-Assessment of Learning: Total Marks (as per your learning and coding skills)**

ASSIGNMENT 4 PART THREE WILL NOT BE GRADED WITHOUT THIS TITEL PAGE
PLEASE FILL ALL ITEMS (-25 % MARKS FOR EACH MISSED ITEM)

HANDWRITTEN PART: SELF-ASSEMENT TABLE (must be filled to get marks in this assignment) [fill for each problem and tasks]

Problem and Task Number	% Completed, Compiles and Runs	Reason for not coding	Coding Skills	Marks	Justification of marks

Please compile and run each program before filling the self-assessment Table:

Program Compiles and Runs is 50 % Marks.

Program Completely Coded is 50 % Marks.

See the following example table on how to fill the above table: [give 10 marks for each completed program that is submitted on portal)

Problem Number	% Completed, Compiles and Runs	Reason for not coding	Coding Skills	Marks	Justification of marks
1 - 3	0	Very confident of my skills for these problems	Array and Stack Implementation and Application	5/5	Can Solve problmes using Array and Stack, can code without help in evaluation by the teacher
4 - 9	100 completed, compiles and runs	NA	All Types of Queues	30/30	Know and can code any Queue implementation and application problem and

					confident of evaluation
10 - 13	100 completed, 50 % compiles and runs	NA	All Types of LinkedList	20/40	Not able to remove compile and run-time erros

Portal Submission Instructions

Please Submit **one code file** for each complete program(**must use the following provided template).** **Therefore, submit as many .cpp files as many complete programs. Any ZIP or RAR or any other type of file is zero marks.**

Example Template File for A3 P5:

```
// SPRING 2026 DATA STRUCTURES SECTION D20 ASSIGNMENT THREE
// Name of Student:
// Registration of Student:
// File A3_PROBLEM_#.cpp contains complete code of THE problem , that will compile
and run

/***** GLOBAL FUNCTION PROTOTYPES *****/
// Write PROTOTYPES of All Global Functions Here

/***** write in UPPERCASE the name of CLASS ONE *****/
// Declaration (prototype) of Class One
// Definition (Code) of Class One
/***** write in UPPERCASE the name of CLASS TWO *****/
// Declaration (prototype) of Class Two
// Definition (Code) of Class Two
---------------------------------------------
---------------------------------------------
/***** write in UPPERCASE the name of LAST CLASS*****/
// Declaration (prototype) of Class Last
// Definition (Code) of Class Last

/***** GLOBAL FUNCTION CODE *****/
// Write Codes of All Global Functions Here

/***** Driver Function Main *****/
// CODE OF main here
```

Program Compiles and Runs is 50 % Marks.

Program Completely Coded is 50 % Marks.

ADT HEADER FILES

Header files to be used in this assignment are posted on the Portal as a zip file. You must use only these header files without any change.

If you need to modify the above ADT, you must provide a conclusive reason of why the following functions can't be coded without the modification

ADT 1: Array

1. Complete the code of **ADT Array** and write a function main to test the class.
2. **Use the above ADT Class to write and test, using a main, the following GLOBAL functions (**must use the provided function prototypes). **Of course, you will need to provide a concrete class for the above ADT.**

    a) copy: that return the copy of the array:

    void copy(AbstarctArray source, AbstarctArray & destination);

    b) compare: that compare two arrays for equality

    bool copy(AbstarctArray A, AbstarctArray B);

    c) delete: that deletes number of values from a given index by shifting left

    void delete(AbstarctArray values, const int & noOfValues, const int & index);

    d) insert: that inserts one array in another array at a given index by shifting destination array to right. (Values greater than the array size are cut-off)

    void insert(AbstarctArray source, AbstarctArray & destination, const int & index);

    e) mode: that calculates and return the frequency of each value of the Array
    AbstractArray frequency(AbstarctArray source);

ADT 2: Stack

3. Complete the code of **ADT Stack** and write a function main to test the class
4. **Use the above ADT Class to write and test, using a main, the following GLOBAL functions (**must use the provided function prototypes). **Of course, you will need to provide a concrete class for the above ADT. (Please Use AbstractArray Based Stack)**

    a) copy: that return the copy of the Stack:

    void copy(Stack source, Stack & destination);

    b) compare: that compare two Stacks for equality

bool copy(Stack A, Stack B);

c) delete: that deletes number of values from a given index by shifting downward

void delete(Stack values, const int & noOfValues, const int & index);

d) insert: that inserts one Stack in another Stack at a given position by shifting destination Stack Up.

void insert(Stack source, Stack & destination, const int & index);

e) mode: that calculates and return the frequency of each value of the Stack
Stack frequency(Stack source);

5. Use **only the Stack ADT**, please Write the following Complete C++ Programs using **GLOBAL** Functions that use **Public Functions of Stack**
   a) Read an integer from the user, separate the individual digits and store in a Stack, check if this integer is a Palindrome using only TWO Stacks
   b) Read text from a text file "text.txt" and store in a Stack character by character, now calculate the frequencies of each character and display the characters and the frequency of each character

ADT 3: Queue

6. Complete the code of **ADT Queue** and write a function main to test the class
7. **Use the above ADT Class to write and test, using a main, the following GLOBAL functions (**must use the provided function prototypes). **Of course, you will need to provide a concrete class NonCircularQueue for the above ADT. (Please Use AbstractArray Based Queue)**

   a) copy: that return the copy of the Queue:

   void copy(Queue source, Queue & destination);

   b) compare: that compare two Queues for equality

   bool copy(Queue A, Queue B);

   c) delete: that deletes number of values from a given index

   void delete(Queue values, const int & noOfValues, const int & index);

   d) insert: that inserts one Queue in another Queue at a given position

   void insert(Queue source, Queue & destination, const int & index);

   e) mode: that calculates and return the frequency of each value of the Queue
   Queue frequency(Queue source);

8. Use **only the Queue ADT**, please Write the following Complete C++ Programs using **GLOBAL** Functions that use **Public Functions of Queue**
   a) Read an integer from the user, separate the individual digits and store in a Queue, check if this integer is a Palindrome.

b) **Read text from a text file "text.txt" and store in a Queue character by character, now calculate the frequencies of each character and display the characters and the frequency of each character**

9. **Repeat Problems 4 using a CircularQueue (Modify the ADT Queue accordingly)**
10. **Repeat Problems 5 using a CircularQueue (Modify the ADT Queue accordingly)**

ADT 4: LinkedList

11. Complete the code of **ADT LinkedList** and write a function main to test the class

12. **Use the above ADT Class to write and test, using a main, the following GLOBAL functions (**must use the provided function prototypes). **Of course, you will need to provide a concrete class MyLinkedList for the above ADT.**

    a) copy: that return the copy of the LinkedList:

    void copy(LinkedList source, LinkedList & destination);

    b) compare: that compare two LinkedLists for equality

    bool copy(LinkedList A, LinkedList B);

    c) delete: that deletes number of values from a given index

    void delete(LinkedList values, const int & noOfValues, const int & index);

    d) insert: that inserts one LinkedList in another LinkedList at a given position

    void insert(LinkedList source, LinkedList & destination, const int & index);

    e) mode: that calculates and return the frequency of each value of the LinkedList
    LinkedList frequency(LinkedList source);

13. Use **only the LinkedList ADT**, please Write the following Complete C++ Programs using **GLOBAL** Functions that use **Public Functions of LinkedList**
    a) Read an integer from the user, separate the individual digits and store in a LinkedList, check if this integer is a Palindrome.
    b) Read text from a text file "text.txt" and store in a LinkedList character by character, now calculate the frequencies of each character and display the characters and the frequency of each character

14. **Repeat Problems 4 & 5 using a SortedLinkedList**
15. **Repeat Problems 4 & 5 using a FirstOnlyCircularLinkedList**
16. **Modify he class node to also have previous pointer, now complete the implementation of DoublyLinkedList and write a main function to test DoublyLinkedList**

```
class Node
{
 public:
   int data;
```

```
    Node *next;
    Node *previous;
};
```

17. Repeat Problems 4 & 5 using a DoublyLinkedList